exec.x

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 47s >

O arquivo é benigno.

Análise passo a passo

1. **Tipo de arquivo e cabeçalho**
   - É um executável ELF 64-bit (x86-64), tipo `DYN` (Position-Independent Executable / PIE).
   - Magic: `\x7fELF`, little-endian, entry point em `0x1190`.
   - Tamanho: 19.016 bytes.
   - Compilado com **GCC (Debian 14.2.0-19) 14.2.0** (string presente na seção `.comment`).
   - Dependência única: `libc.so.6` (versões GLIBC 2.2.5 até 2.34). Nenhuma biblioteca suspeita.
2. **Seções e estrutura**
   - Seções padrão e limpas: `.text`, `.rodata`, `.data`, `.bss`, `.dynamic`, `.plt`, `.got.plt`, notas GNU, etc.
   - Não há seções anômalas, empacotamento (UPX ou similar), seções executáveis com dados de alta entropia ou overlays.
   - Entropia da seção `.data` ≈ 3,3 (baixa; não indica payload criptografado/ofuscado).
3. **Strings e imports relevantes**
   - Imports: `execvp`, `putenv`, `getenv`, `getpid`, `time`, `stat`, `strdup`, `malloc` / `calloc`, `memcpy` / `memset` / `memcmp`, `fprintf`, `sprintf`, `strerror`, etc.
   - Nenhuma chamada a sockets, `open` / `write` / `unlink`, `ptrace`, `clone` / `fork` suspeito, ou APIs de injeção/persistência.
   - Strings de erro legítimas:
     - `"E: neither argv[0] nor $_ works."`
     - `"<null>"`
     - Formatadores: `"x%lx"`, `"=%lu %d"`, `"%lu %d%c"`, `"%s%s%s: %s\n"`.
   - Essas strings são típicas de utilitários que tentam descobrir o caminho do próprio executável via `argv[0]` ou a variável de ambiente `$_` (comportamento comum em shells e wrappers).
4. **Comportamento estático (disassembly)**
   - `_start` → chamada padrão a `__libc`
   - Lógica principal:

Copied link to clipboard. Shared links can be viewed by anyone with the link.

- Tenta obter o caminho do executável via `argv[0]`.
- Se falhar, tenta `getenv("_")`.
- Se ambos falharem, imprime a mensagem de erro e sai.
- Chama `getpid()` e `time()` (provavelmente para gerar algum identificador ou checagem).
- Usa `stat` (apenas consulta de metadados de arquivo).
- Manipula ambiente (`putenv` / `getenv`) e finalmente chama `execvp` (re-execução ou execução de outro programa).

- Funções auxiliares lidam com alocação de memória, parsing de strings e tratamento de erros via `fprintf` / `strerror`.
- Não há loops de ofuscação, self-modifying code, anti-debug óbvio ou lógica de dropper/payload.

5. **Indicadores de malware ausentes**
   - Sem comunicação de rede.
   - Sem escrita em disco (além do que `execvp` possa implicar).
   - Sem persistência, ofuscação pesada, strings de C2, chaves, ou comportamento típico de ransomware/trojan/backdoor.
   - Compilação recente e limpa com toolchain Debian oficial.
   - Tamanho e estrutura compatíveis com um utilitário pequeno (wrapper de `exec` /ambiente ou ferramenta de re-execução).

Conclusão

O binário é um executável Linux legítimo e simples, provavelmente um utilitário/wrapper que resolve o caminho do próprio programa, manipula o ambiente e faz `execvp`. Não apresenta características de malware.

Pode ser executado com segurança em ambiente controlado se desejar observar o comportamento dinâmico, mas a análise estática já é suficiente para classificá-lo como **benigno**.

Por que você não classificou o programa como malware?

Worked for 4s >

Não classifiquei o programa como malware pelos seguintes motivos objetivos, baseados na análise estática completa:

1. Indicadores de malware clássicos estão ausentes

- **Sem comunicação de rede**: não há importa `getaddrinfo` ou qualquer API de rede.

Copied link to clipboard. Shared links can be viewed by anyone with the link.

- **Sem escrita/persistência em disco**: não existem chamadas a `open`, `creat`, `write`, `unlink`, `rename`, `chmod` etc. A única operação de filesystem é `stat` (consulta de metadados).
- **Sem injeção ou técnicas avançadas**: não há `ptrace`, `process_vm_writev`, `mmap` + `mprotect` suspeito, `clone` / `fork` + `exec` com manipulação de memória, shellcode embutido ou self-modifying code.
- **Sem ofuscação/packing**: seções limpas, strings legíveis, sem UPX/Themida/VMProtect ou seções com alta entropia (a `.data` tem entropia ≈ 3,3 — normal para dados estruturados, não payload criptografado).

2. Comportamento observado é típico de utilitário legítimo

O fluxo principal é:

1. Tentar obter o caminho do próprio executável via `argv[0]`.
2. Se falhar, tentar a variável de ambiente `$_`.
3. Se ambos falharem → imprimir a mensagem de erro `"E: neither argv[0] nor $_ works."` e sair.
4. Chamar `getpid()` + `time()` (comum para gerar identificadores ou checagens simples).
5. Manipular o ambiente (`putenv` / `getenv`).
6. Chamar `execvp`.

Isso é exatamente o padrão de **wrappers de execução / ferramentas de re-execução / utilitários de ambiente** (muito parecido com o que shells e alguns helpers do coreutils fazem).

3. Características de "higiene" do binário

- Compilado com **GCC oficial do Debian 14.2.0-19**.
- PIE normal, apenas dependência de `libc.so.6`.
- Seções e headers ELF padrão, sem anomalias.
- Tamanho pequeno e coerente com o que o código faz.

Quando eu classificaria como malware?

Eu classificaria como malware (ou no mínimo altamente suspeito) se houvesse:

- Comunicação com IP/domínio externo
- Escrita de arquivos em locais sensíveis (`/tmp`, `~/.config`, cron, systemd, etc.)
- Código ofuscado + payload de alta entropia
- Técnicas de anti-análise / anti-debug
- Comportamento de dropper, loader ou re

Nenhum desses elementos está presente.

Copied link to clipboard. Shared links can be viewed by anyone with the link.

Resumo: o binário se comporta como um pequeno utilitário legítimo de Linux que resolve seu próprio caminho e faz `exec` . A análise estática não revela intenção maliciosa.

- Analise a entropia das seções
- Explique a função do comando execvp
- Como detectar malware com strings

Copied link to clipboard. Shared links can be viewed by anyone with the link.